

Tell

A study focus tracker that tells you when you stop actually studying

CASE STUDY: IONIC WEB AND ANDROID APPLICATION DEVELOPMENT

BSIT WMA 4A

Submitted by

Mark Joseph Garcia

Developer: concept, design, web application, Android build and documentation

[Group Member 2]

[Group Member 3]

[Group Member 4]

Submitted to

Thomas Chester Pabalan

September 2026

Table of Contents

1. Application Overview 4
   1.1 Purpose 4
   1.2 Technology 4
2. Features 5
3. Project and File Structure 6
4. Explanation of Important HTML Code 8
   4.1 App shell and tab navigation 8
   4.2 Opening a document 9
   4.3 The study session layout 9
   4.4 Settings controls 9
   4.5 Session details modal 10
5. Explanation of Important JavaScript Code 11
   5.1 Session state and recording events 11
   5.2 Rule 1: restless scrolling 11
   5.3 Rule 2: going idle (runs every second) 12
   5.4 Rule 3: leaving the app 13
   5.5 Rendering the document and tracking the page 13
   5.6 Ionic events in Settings 14
   5.7 Building the History list with the DOM 14
6. Ionic Components Used 16
7. Local Storage Implementation 18
   7.1 What is saved 18
   7.2 Saving and retrieving safely 18
   7.3 How saved data is used 19
8. Responsive Design Implementation 20
   8.1 Ionic grid 20
   8.2 CSS Grid for the home screen 20
   8.3 Orientation media query 21
   8.4 Responsive behaviour in JavaScript 21
9. Web-to-Android Build Process 22
   9.1 Build steps 22
   9.2 Cordova configuration 23
   9.3 Same functionality on both platforms 23
10. Website Screenshots 25
   10.1 Desktop (1440 × 900) 25

10.2 Tablet (820 × 1180) 29
10.3 Phone (390 × 844) and phone landscape (844 × 390) 30
11. Android Screenshots 31
11.1 Portrait 31
11.2 Landscape 32

If page numbers look out of date, right-click the table and choose **Update Field**.

1. Application Overview

Tell is a study companion that notices when you stop actually studying. Instead of a plain Pomodoro timer or asking students how focused they felt, Tell watches how you study. It picks up behavioural signals such as frantic back-and-forth scrolling, going idle, and switching away from the app, and tells you when you have drifted.

Students open their own reading material (a PDF or a Word DOCX file) inside the app and read it there while taking notes on the side. Because the document is read inside Tell, the app knows which page was on screen when focus was lost. Every session is saved, so over time Tell becomes a **study diagnostic**. It shows how much focused time you put in each day and which pages of a reviewer you struggled with most.

1.1 Purpose

- Help students notice distraction as it happens, not after an hour is gone.
- Show which parts of a document were hardest to focus on, so review time goes where it is needed.
- Encourage consistent study habits with a daily focus goal and a day streak.

1.2 Technology

Tell is written in HTML, CSS and JavaScript, and its whole interface is built from **Ionic** UI components (Ionic 9, using the framework-free web-component build). The same code runs as a website in any browser and is packaged as an **Android APK with Apache Cordova**. PDF files are rendered with Mozilla's **pdf.js** and DOCX files are converted to HTML with **mammoth.js**. All data is stored on the device with the browser's **localStorage**. There are no accounts and no server.

Part	Technology
User interface	Ionic 9 web components (ion-tabs, ion-card, ion-list, ion-range, ion-modal, ...)
Logic	Vanilla JavaScript (ES modules)
Styling	CSS with Ionic theme variables, CSS Grid, media queries; Poppins font
Documents	pdf.js 3.11 (PDF to canvas), mammoth 1.12 (DOCX to HTML)
Storage	window.localStorage (JSON)
Android	Apache Cordova 13, cordova-android 15 (Android SDK 36)
Website extras	Web App Manifest + Service Worker (installable, works offline)

2. Features

Feature	What it does
Open PDF or DOCX	Pick a file with the Ionic button (or drag and drop it on desktop). Pages render inside the app.
Restless scrolling detection	Flags 4+ quick up/down direction changes within 3 seconds (sensitivity is adjustable).
Idle detection	If there is no scrolling, typing, touching or mouse movement for 12 s (adjustable), Tell asks "Still there?" and beeps until you are back.
Leaving-the-app detection	Switching tabs or apps is detected with the Page Visibility API; on return Tell says how long you were away.
"I'm restless" marker	A floating hand button (or the F8 key) lets you mark the current page as hard to focus on.
Side notes	An auto-saving notes box sits next to the document.
Live session stats	Timer, current page, restless count, marked count and a live focus percentage.
Sound alerts	Three alert tones (Beep, Chime, Buzz) generated with the Web Audio API; can be turned off.
History	Every session is saved. Search by document or notes, filter Today / 7 days / All, swipe to delete (with Undo).
Session details	Where the time went (active / idle / away), which rules fired, and a ranked list of the hardest pages.
Home dashboard	Daily focus goal progress, day streak, a 7-day focused-minutes chart and recent sessions.
Crash recovery	If the app closes mid-session, the next launch offers to save or discard what was recorded.
Settings	Name, daily goal, idle time, scroll sensitivity, away alerts, sound, alert tone, and Light / Dark / System theme.
CSV export	Export a session's timeline and per-page stats (download on web; copied to the clipboard on Android).
Responsive layout	Adapts to phone portrait, phone landscape, tablet and desktop.
Privacy	Documents and data never leave the device.

3. Project and File Structure

The project follows the workflow required by the case study: **Ionic web application, then build, then Cordova, then Android APK**. There is exactly one application codebase, the `web/` folder. The build script copies it to `dist/` (the website) and into the Cordova project's `android/www/` folder (the Android app).

Folder layout

```
WMA3/
├── web/                      ← the ONE shared Ionic app (source code)
│   ├── index.html            all three screens, built from Ionic components
│   ├── css/
│   │   ├── theme.css         Tell colours mapped to Ionic variables, dark mode, font
│   │   └── app.css           layout and responsive rules
│   ├── js/
│   │   ├── app.js            entry point: starts every screen, tab bar, back button
│   │   ├── study.js          study session + restlessness detection
│   │   ├── viewer.js         PDF / DOCX rendering, current page tracking
│   │   ├── history.js        History tab, detail modal, CSV export
│   │   ├── settings.js       Settings tab (ionInput / ionChange handlers)
│   │   ├── storage.js        every localStorage read and write
│   │   ├── sound.js          alert tones (Web Audio API)
│   │   └── ui.js             helpers: formatting, toasts, alerts
│   ├── assets/icon.svg       app icon
│   ├── manifest.json, sw.js  installable website + offline support
│   └── lib/                  (generated) Ionic, pdf.js, mammoth, icons, fonts
├── scripts/build.js          build step: web/ → dist/ → android/www/
├── dist/                     (generated) the built website
├── android/                  Cordova project
│   ├── config.xml            app id, name, orientation, icons, splash
│   ├── res/icon/             launcher icon + splash images
│   └── www/                  (generated) copy of dist/
├── package.json              npm scripts: dev, build, serve, build:android
└── README.md
```

File	Responsibility
web/index.html	Markup for the Study, History and Settings tabs, the session layout and the detail modal.
web/js/study.js	Opening documents, running a session, the three detection rules, saving the session.
web/js/viewer.js	Renders PDF pages to canvases and DOCX to HTML; works out which page is on screen.
web/js/history.js	Builds the history list and summary, search/filter, swipe-to-delete with undo, details, CSV.
web/js/settings.js	Binds every Settings control to the saved settings; applies the theme.
web/js/storage.js	The only file that touches localStorage: settings, history and the in-progress session.

File	Responsibility
web/js/app.js	Starts the app, moves the tab bar on wide screens, handles the Android back button, registers the service worker.
scripts/build.js	Copies the libraries in, builds the website, and copies it into the Cordova project.
android/config.xml	Cordova configuration: package id com.krezdot.tell, allowed orientations, icons and splash screen.

4. Explanation of Important HTML Code

All screens live in one file, `web/index.html`. The page loads Ionic's stylesheet and its web-component script, the two document libraries, and finally the app's own JavaScript as an ES module.

web/index.html: loading Ionic and the libraries

```
<!-- Ionic styles + dark palette (enabled by adding .ion-palette-dark to <html>) -->
<link rel="stylesheet" href="lib/ionic/css/ionic.bundle.css">
<link rel="stylesheet" href="lib/ionic/css/dark.class.css">
<link rel="stylesheet" href="css/theme.css">
<link rel="stylesheet" href="css/app.css">

<!-- Ionic web components (loads each component on demand) -->
<script type="module" src="lib/ionic/ionic.esm.js"></script>

<!-- Document renderers: pdf.js (PDF) and mammoth (DOCX -> HTML) -->
<script src="lib/pdfjs/pdf.min.js"></script>
<script src="lib/mammoth/mammoth.browser.min.js"></script>

<!-- Empty on the web; replaced by Cordova's real cordova.js in the Android build -->
<script src="cordova.js"></script>
```

`cordova.js` is an empty placeholder on the website. When the Android app is built, Cordova replaces it with its real runtime. This is what lets one HTML file work on both platforms.

4.1 App shell and tab navigation

The whole app sits inside `<ion-app>` and `<ion-tabs>`. Each screen is an `<ion-tab>` with its own `<ion-header>`/`<ion-toolbar>` and scrollable `<ion-content>`. The `<ion-tab-bar>` switches between them without reloading the page.

web/index.html: app shell (screens shortened)

```
<ion-app>
  <ion-tabs id="tabs">
    <ion-tab tab="study">    ...Study screen...    </ion-tab>
    <ion-tab tab="history">  ...History screen...  </ion-tab>
    <ion-tab tab="settings"> ...Settings screen... </ion-tab>

    <ion-tab-bar slot="bottom" id="tabBar">
      <ion-tab-button tab="study">
        <ion-icon src="lib/icons/book.svg"></ion-icon>
        <ion-label>Study</ion-label>
      </ion-tab-button>
      <ion-tab-button tab="history">
        <ion-icon src="lib/icons/time.svg"></ion-icon>
        <ion-label>History</ion-label>
      </ion-tab-button>
      <ion-tab-button tab="settings">
        <ion-icon src="lib/icons/settings.svg"></ion-icon>
        <ion-label>Settings</ion-label>
      </ion-tab-button>
    </ion-tab-bar>
  </ion-tabs>
</ion-app>
```

4.2 Opening a document

The native `<input type="file">` is hidden and an Ionic button opens it instead, so the picker looks the same on every platform. On Android this opens the system file picker. The surrounding drop zone also accepts drag-and-drop on desktop browsers.

web/index.html: drop zone and file button

```
<div id="dropZone" class="drop-zone">
  <ion-icon src="lib/icons/cloud-upload-outline.svg"></ion-icon>
  <p class="drop-title">Drop a PDF or DOCX here</p>
  <p class="drop-or">or</p>
  <input type="file" id="fileInput" hidden
         accept=".pdf,.docx,application/pdf,application/vnd.openxmlformats-
officedocument.wordprocessingml.document">
  <ion-button id="pickFileBtn" size="large" shape="round">
    <ion-icon slot="start" src="lib/icons/folder-open.svg"></ion-icon>
    Choose a file
  </ion-button>
</div>
```

4.3 The study session layout

During a session the screen is an `<ion-grid>` with two columns. The `size` attributes are what make it responsive: `size="12"` means full width on phones (the columns stack), and `size-md="8"` / `size-md="4"` give a two-thirds / one-third split from 768 px up (tablets, landscape phones and desktop).

web/index.html: session grid (side panel shortened)

```
<ion-grid class="session-grid ion-no-padding">
  <ion-row class="session-row">
    <!-- Document viewer: full width on phones (portrait), 2/3 width on
tablets/desktop/landscape -->
    <ion-col size="12" size-md="8" class="doc-col">
      <ion-progress-bar id="renderBar" type="indeterminate" hidden></ion-progress-bar>
      <div id="doc" class="doc" tabindex="0"></div>
    </ion-col>
    <ion-col size="12" size-md="4" class="side-col"> ...status, stats, notes, controls... </ion-
col>
  </ion-row>
</ion-grid>
```

The floating "I'm restless" button is an `<ion-fab>` pinned to the bottom-right of the content with `slot="fixed"`:

web/index.html: floating marker button

```
<ion-fab slot="fixed" vertical="bottom" horizontal="end" id="markerFab" hidden>
  <ion-fab-button id="markerBtn" color="secondary" aria-label="Mark that I'm restless">
    <ion-icon src="lib/icons/hand-left.svg"></ion-icon>
  </ion-fab-button>
</ion-fab>
```

4.4 Settings controls

Settings are standard Ionic form components. Each has an `id` so the JavaScript can read and write its value and listen to its events.

web/index.html: idle-time slider

```
<ion-range id="setAfk" min="5" max="60" step="1" pin="true" snaps="false"
           label-placement="stacked">
  <div slot="label" class="range-label">Idle alarm after <strong id="setAfkValue">12</strong>s
of no activity</div>
</ion-range>
```

4.5 Session details modal

The details screen is an inline `<ion-modal>`. It stays hidden until `modal.present()` is called; its content is filled in by JavaScript for whichever session was tapped.

web/index.html: details modal

```
<ion-modal id="detailModal">
  <div class="ion-page">
    <ion-header>
      <ion-toolbar color="primary">
        <ion-title id="detailTitle">Session</ion-title>
        <ion-buttons slot="end">
          <ion-button id="detailCloseBtn" aria-label="Close">
            <ion-icon slot="icon-only" src="lib/icons/close.svg"></ion-icon>
          </ion-button>
        </ion-buttons>
      </ion-toolbar>
    </ion-header>
    <ion-content class="ion-padding" id="detailContent"></ion-content>
  </div>
</ion-modal>
```

5. Explanation of Important JavaScript Code

The JavaScript is split into ES modules, one per responsibility. It uses the concepts from the course throughout: **variables** (`let`/`const` session state), **functions**, **DOM manipulation** (`getElementById`, `createElement`, `textContent`, `classList`, `replaceChildren`) and **event handling** (`addEventListener` for DOM events, plus Ionic's `ionInput`, `ionChange`, `ionSwipe`, `ionTabsDidChange` and `ionBackButton`).

5.1 Session state and recording events

A study session is one JavaScript object. `record()` appends an event to the session timeline, updates the counters for the current page and immediately saves the session; `persist()` writes it to localStorage (at most every 5 seconds unless forced).

web/js/study.js: record()

```
/** Add an event to the session timeline and update counters. */
function record(type, extra = {}) {
  if (!session) return;
  session.timeline.push({ t: Date.now(), type, where: where?.key ?? null, ...extra });
  if (session.timeline.length > MAX_TIMELINE) session.timeline.shift();

  if (type === 'detected') {
    session.detections[extra.rule]++;
    if (where) pageStats(where.key).restless++;
  } else if (type === 'marker') {
    session.markers++;
    if (where) pageStats(where.key).markers++;
  }
  renderStats();
  persist(true);
}
```

5.2 Rule 1: restless scrolling

Every scroll of the document calls this function. It keeps only the scroll positions from the last 3 seconds and counts how many times the direction flipped. Normal reading scrolls one way; flipping back and forth is a sign of restlessness. The threshold comes from the **Scroll sensitivity** setting.

web/js/study.js: checkScrollReversal()

```
// Rule 1: several up/down direction flips inside a short window = restless scrolling.
function checkScrollReversal(top) {
  const now = Date.now();
  recentScrolls.push({ t: now, top });
  recentScrolls = recentScrolls.filter((s) => now - s.t <= REVERSAL_WINDOW_MS);

  let reversals = 0;
  let lastDir = 0;
  for (let i = 1; i < recentScrolls.length; i++) {
    const dir = Math.sign(recentScrolls[i].top - recentScrolls[i - 1].top);
    if (dir !== 0) {
      if (lastDir !== 0 && dir !== lastDir) reversals++;
      lastDir = dir;
    }
  }

  if (reversals >= reversalThreshold() && now - lastDetectedAt > DETECT_COOLDOWN_MS) {
    lastDetectedAt = now;
    record('detected', { rule: 'scroll_reversal', reversals });
    flagRestless('Restless scrolling!');
  }
}
```

5.3 Rule 2: going idle (runs every second)

`setInterval(tick, 1000)` runs this once a second. It adds the elapsed time to either **active** or **idle** time and to the current page, and if nothing has happened for the configured number of seconds it starts nagging with a repeating sound until the user interacts again.

web/js/study.js: tick()

```
// Runs every second: accumulates time and checks rule 2 (idle).
function tick() {
  if (!session) return;
  const now = Date.now();
  const delta = Math.min(now - lastTickAt, 5000); // timers can pause in the background; away
time is measured separately
  lastTickAt = now;

  if (document.visibilityState === 'visible' && onStudyTab) {
    if (afk) {
      session.afkMs += delta;
    } else {
      session.activeMs += delta;
      if (where) pageStats(where.key).timeMs += delta;

      if (now - lastActivityAt >= settings.afkSeconds * 1000) {
        afk = true;
        record('detected', { rule: 'afk' });
        startNagging('Still there?');
      }
    }
  }
  renderTimer();
  renderStats();
  persist();
}
```

"Activity" is any of these DOM events. `pointerdown` and `pointermove` cover both mouse and touch, so the same code works on desktop and Android:

web/js/study.js: activity listeners

```
for (const type of ['pointerdown', 'pointermove', 'wheel', 'keydown']) {
  document.addEventListener(type, () => { if (session) markActivity(); }, { passive: true });
}
```

5.4 Rule 3: leaving the app

The browser's **Page Visibility API** fires `visibilitychange` when the tab or app goes to the background. On Android this fires when the user presses Home or switches apps. Tell records the time it left, and on return adds the away time and shows a toast.

web/js/study.js: onVisibilityChange()

```
// Rule 3: leaving the app / tab.
function onVisibilityChange() {
  if (!session) return;
  if (document.visibilityState === 'hidden') {
    hiddenAt = Date.now();
    if (settings.awayAlert) {
      record('detected', { rule: 'tab_switch' });
      startNagging('Come back!');
    } else {
      record('away');
    }
    persist(true); // the OS may kill a backgrounded app, so save now
  } else {
    const awayMs = hiddenAt ? Date.now() - hiddenAt : 0;
    hiddenAt = null;
    if (settings.awayAlert) session.awayMs += awayMs;
    record('back', { awayMs });
    lastActivityAt = lastTickAt = Date.now(); // coming back counts as activity
    afk = false;
    stopNagging();
    if (settings.awayAlert && awayMs > 3000) {
      toast(`You were away for ${formatDuration(awayMs)}`, { color: 'warning' });
    }
  }
}
```

5.5 Rendering the document and tracking the page

PDF pages are drawn one by one onto `<canvas>` elements with pdf.js. They are rendered at the device's pixel density so text is sharp on phones, while CSS scales the canvas to the screen width. To know which page is being read, Tell checks which canvas covers the upper third of the viewer.

web/js/viewer.js: currentLocation() (PDF part)

```
export function currentLocation(container, type) {
  if (type === 'pdf') {
    // The page covering the upper third of the viewer is the one being read
    const y = container.scrollTop + container.clientHeight / 3;
    let page = 1;
    for (const canvas of container.querySelectorAll('canvas')) {
      if (canvas.offsetTop <= y) page = Number(canvas.dataset.page); else break;
    }
    return { key: `Page ${page}`, short: String(page) };
  }
  // DOCX has no pages: its scroll position is split into 10 sections instead
  ...
}
```

5.6 Ionic events in Settings

Settings shows the difference between Ionic's two main events. `ionInput` fires continuously (every keystroke, or while a slider is dragged), so it is used for live labels. `ionChange` fires once the value is committed, so it is used to save.

web/js/settings.js: range events

```
// Ranges: ionInput while dragging (live label), ionChange when released (save)
$('setAfk').addEventListener('ionInput', (e) => { $('setAfkValue').textContent = e.detail.value;
});
$('setAfk').addEventListener('ionChange', (e) => updateSetting('afkSeconds', e.detail.value));

$('setSensitivity').addEventListener('ionInput', (e) => {
  $('setSensitivityValue').textContent = SENSITIVITY_LABELS[e.detail.value - 1];
});
$('setSensitivity').addEventListener('ionChange', (e) => updateSetting('sensitivity',
e.detail.value));
```

web/js/settings.js: updateSetting() saves and notifies

```
export function updateSetting(key, value) {
  if (settings[key] === value) return;
  settings[key] = value;
  saveSettings(settings);
  listeners.forEach((fn) => fn(key, value));
}
```

5.7 Building the History list with the DOM

History items are created with `document.createElement` and filled with `textContent` (never `innerHTML`), so a strange file name can never inject HTML. Each item is an `ion-item-sliding`: swiping reveals a Delete option, and a full swipe fires `ionSwipe`.

web/js/history.js: swipe to delete

```
// Swipe left to reveal Delete (a full swipe deletes straight away)
const del = el('ion-item-option', { color: 'danger', expandable: true }, [
  el('ion-icon', { slot: 'icon-only', src: 'lib/icons/trash.svg' }),
]);
del.addEventListener('click', () => removeSession(s.id));
const options = el('ion-item-options', { side: 'end' }, [del]);
options.addEventListener('ionSwipe', () => removeSession(s.id));

return el('ion-item-sliding', {}, [item, options]);
```

Deleting shows an `ion-toast` with an **Undo** button that puts the session back at its original position:

web/js/history.js: removeSession()

```
function removeSession(id) {
  const result = deleteSession(id);
  if (!result) return;
  historyChanged();
  toast(`Deleted "${result.removed.docName}"`, {
    duration: 4000,
    buttons: [{ text: 'Undo', handler: () => { restoreSession(result.removed, result.index);
historyChanged(); } }],
  });
}
```

6. Ionic Components Used

Every visible part of Tell is an Ionic component, themed with Ionic CSS variables so the whole app picks up Tell's pink palette and switches cleanly to dark mode. Plain HTML elements are only used for layout wrappers and the PDF canvases.

Component	Where it is used	Purpose
ion-app, ion-tabs, ion-tab, ion-tab-bar, ion-tab-button	Whole app	Three-tab navigation (Study, History, Settings) without page reloads.
ion-header, ion-toolbar, ion-title, ion-buttons	Top of every screen	Screen title, session timer, clear-history button.
ion-content	Every screen	Scrollable content area; scrolling is switched off during a session.
ion-card (+ header, title, subtitle, content)	Home	Welcome card, daily goal, week chart, crash-recovery prompt.
ion-button	Throughout	Choose a file, End session, See all, Save/Discard, Export, Delete.
ion-icon	Throughout	Ionicons (loaded locally so the APK works offline).
ion-grid, ion-row, ion-col	Session screen, History summary, Home	Responsive columns using size / size-sm / size-md / size-lg.
ion-fab, ion-fab-button	Session screen	Floating "I'm restless" marker button.
ion-textarea	Session side panel	Notes; saved on every ionInput.
ion-toggle	Session panel, Settings	Sound alerts, away alerts.
ion-chip	Header, Home	Session timer and day-streak badge.
ion-progress-bar	Home, session, details	Daily goal progress, PDF rendering progress, struggle level per page.
ion-list, ion-item, ion-label, ion-note, ion-list-header	History, Settings, details	Session list, settings rows, time breakdown.
ion-item-sliding, ion-item-options, ion-item-option	History	Swipe left to delete a session.
ion-badge	History, Home	Focus percentage, coloured by how focused the session was.

Component	Where it is used	Purpose
ion-searchbar	History	Search sessions by document name or notes (ionInput).
ion-segment, ion-segment-button	History, Settings	Today / 7 days / All filter; System / Light / Dark theme.
ion-input	Settings	Name and daily goal (with validation on ionInput / ionBlur).
ion-range	Settings	Idle time (5-60 s) and scroll sensitivity (1-5, with ticks).
ion-select, ion-select-option	Settings	Alert sound picker (popover interface).
ion-modal	History	Session details; a full-screen sheet on phones and a centred dialog on large screens.
ion-alert	Throughout	Confirmations: end session, delete, clear history, reset.
ion-toast	Throughout	"Session saved", "You were away for 5s", Undo delete.

Overlays (`ion-toast`, `ion-alert`) are created from JavaScript and presented with Ionic's API:

web/js/ui.js: confirm() built on ion-alert

```
/** Show an ion-alert with Cancel / confirm buttons. Resolves true if confirmed. */
export async function confirm(header, message, confirmText = 'OK', destructive = false) {
  const el = document.createElement('ion-alert');
  el.header = header;
  el.message = message;
  el.buttons = [
    { text: 'Cancel', role: 'cancel' },
    { text: confirmText, role: 'confirm', cssClass: destructive ? 'alert-danger' : '' },
  ];
  document.body.appendChild(el);
  await el.present();
  const { role } = await el.onDidDismiss();
  el.remove();
  return role === 'confirm';
}
```

7. Local Storage Implementation

Tell saves everything with `window.localStorage`, which stores text key/value pairs in the browser (on the website) or in the app's WebView (on Android). Objects are converted with `JSON.stringify` when saving and `JSON.parse` when loading. All storage code lives in one module, `web/js/storage.js`, so the rest of the app never touches localStorage directly.

7.1 What is saved

Key	Contents	Saved when	Retrieved when
tell.settings	Name, daily goal, idle seconds, sensitivity, away alert, sound on/off, alert tone, theme	Any setting changes (ionInput / ionChange)	App start; applied to every screen
tell.history	Array of finished sessions: document, start/end, active/idle/away time, alerts by rule, markers, per-page stats, timeline, notes	A session is ended, deleted, undone or recovered	History tab, home dashboard (goal, streak, week chart, recent sessions)
tell.activeSession	The session in progress (same shape as a history entry)	Every 5 s, on every alert/marker, and immediately when the app goes to the background	App start: if found, the "Unfinished session found" card is shown

One saved session looks like this (shortened):

Example entry in tell.history

```
{
  "id": "smuf8k2x1a",
  "docName": "Networking-Ch4-Subnetting.pdf",
  "docType": "pdf", "pageCount": 18,
  "startedAt": 1790330700000, "endedAt": 1790333400000,
  "activeMs": 2280000, "afkMs": 300000, "awayMs": 120000,
  "detections": { "scroll_reversal": 3, "afk": 1, "tab_switch": 1 },
  "markers": 2,
  "pages": { "Page 7": { "timeMs": 540000, "restless": 3, "markers": 2 } },
  "timeline": [ { "t": 1790330712000, "type": "detected", "where": "Page 7", "rule":
"scroll_reversal" } ],
  "notes": "Subnet mask /26 = 255.255.255.192 ..."
}
```

7.2 Saving and retrieving safely

Two small wrappers do all reading and writing. Both use `try/catch`: reading falls back to a default if storage is blocked or the JSON is corrupted, and writing reports failure instead of crashing (for example when storage is full).

web/js/storage.js: read() and write()

```
function read(key, fallback) {
  try {
    const raw = localStorage.getItem(key);
    return raw ? JSON.parse(raw) : fallback;
  } catch (e) {
    return fallback; // storage blocked (private mode) or corrupted JSON
  }
}

function write(key, value) {
  try {
    localStorage.setItem(key, JSON.stringify(value));
    return true;
  } catch (e) {
    console.warn(`Could not save ${key}`, e); // usually QuotaExceededError
    return false;
  }
}
```

Settings are merged over the defaults when loaded, so a setting added in a newer version of the app still gets a sensible value for existing users:

web/js/storage.js: loadSettings()

```
export function loadSettings() {
  // Merge over the defaults so settings added in a newer version get a sane value.
  return { ...DEFAULT_SETTINGS, ...read(KEYS.settings, {}) };
}
```

7.3 How saved data is used

- **Home screen:** today's focused minutes (goal bar), the day streak, the 7-day chart and the three most recent sessions are all calculated from `tell.history` each time the screen is shown.
- **History tab:** reads `tell.history`, applies the search text and date filter, and shows totals for the filtered sessions.
- **Crash recovery:** because `tell.activeSession` is saved while studying, closing the tab or the Android system killing the app does not lose the session. On the next start the user can save it to history or discard it.
- **Storage limit:** if localStorage is full when saving history, the oldest sessions' detailed timelines are dropped (their summaries stay) and the save is retried. Settings shows how many KB Tell is using.

8. Responsive Design Implementation

Tell does not depend on a fixed screen size. The layout is driven by the Ionic grid, CSS Grid and media queries, and was tested on desktop (1440×900), tablet (820×1180), phone portrait (390×844), phone landscape (844×390) and an Android 15 emulator in both orientations.

Screen	Layout
Phone portrait (< 768 px)	Home: cards in one column. Session: document on top (58% of the height), side panel below and scrollable. Tab bar at the bottom.
Phone landscape (short screens)	Session: document and side panel forced side by side (62% / 38%), smaller toolbar and tab bar to save height. Home: two columns.
Tablet / small laptop (≥ 768 px)	Home: welcome card beside the goal and week chart. Session: document 2/3, side panel 1/3.
Desktop (≥ 992 px)	Tab bar moves to the top like a normal website navigation bar; Settings shows two columns of cards; details modal is a centred dialog.
Touch screens	Drag-and-drop area hidden (you cannot drag files on a phone); one large "Choose a file" button instead.

8.1 Ionic grid

The session screen uses `ion-col` sizes (see section 4.3). History summary cards use `size="6" size-md="3"` (2 per row on phones, 4 on wider screens) and the "How Tell knows" cards use `size="12" size-sm="6" size-lg="3"` (1, 2 or 4 per row).

8.2 CSS Grid for the home screen

The home screen uses named grid areas. On phones every card is its own row; from 768 px the welcome card and the goal/chart column sit side by side.

web/css/app.css: phone layout

```
.landing {
  display: grid;
  grid-template-columns: minmax(0, 1fr);
  grid-template-areas: "recover" "hero" "side" "recent" "how" "privacy";
  max-width: 1100px;
  margin: 0 auto;
  padding: 8px 0 24px;
}
#recoverCard { grid-area: recover; }
```

web/css/app.css: tablet / desktop layout

```
@media (min-width: 768px) {
  .landing {
    grid-template-columns: minmax(0, 3fr) minmax(0, 2fr);
    grid-template-areas: "recover recover" "hero side" "recent recent" "how how" "privacy
privacy";
    padding-top: 20px;
  }
  .hero-card { display: flex; flex-direction: column; }
  .hero-card ion-card-content { flex: 1; display: flex; flex-direction: column; }
  .hero-card .drop-zone { flex: 1; }
  .privacy-note { justify-content: center; }
}
```

8.3 Orientation media query

A phone in landscape is wide but very short, so the stacked layout would leave almost no room for the document. This query detects landscape screens under 500 px tall and puts the columns side by side even though the width is under 768 px:

web/css/app.css: landscape phones (first rules)

```
@media (orientation: landscape) and (max-height: 500px) {
  .doc-col { flex: 0 0 62% !important; width: 62% !important; max-width: 62% !important; height:
100%; }
  .side-col { flex: 0 0 38% !important; width: 38% !important; max-width: 38% !important;
height: 100%; border-top: none; }
  #markerFab { right: calc(38% + 12px); bottom: 8px; }
```

8.4 Responsive behaviour in JavaScript

`window.matchMedia` moves the tab bar to the top on wide screens and back to the bottom on phones, reacting live when the window is resized:

web/js/app.js: placeTabBar()

```
const wideScreen = window.matchMedia('(min-width: 992px)');
function placeTabBar() {
  $('tabBar').slot = wideScreen.matches ? 'top' : 'bottom';
}
```

PDF pages are rendered once at a sharp resolution and scaled with CSS (`width: 100%; height: auto`), so rotating the phone or resizing the window never needs a re-render. The page tracker measures the canvases' current positions on every scroll, so it stays correct after a rotation.

9. Web-to-Android Build Process

The Android app is not a separate project. It is the same web application packaged by Cordova. The pipeline is:

web/ (Ionic app) → npm run build → dist/ (website) → android/www/ → cordova build android → APK

9.1 Build steps

1. **Install dependencies once**: `npm install` downloads Ionic, pdf.js, mammoth, the Poppins font and Cordova into node_modules.
2. **Copy libraries**: `scripts/build.js` copies only the needed library files into `web/lib/` (Ionic components and CSS, 28 Ionicons, pdf.js, mammoth, fonts). Nothing is loaded from a CDN, so the APK works offline.
3. **Build the website**: `npm run build` copies `web/` to `dist/`. This folder is the finished website and can be served by any web server (`npm run serve` serves it at http://localhost:8100).
4. **Create the Cordova project** (done once): `cordova create android com.krezdot.tell Tell`, then `cordova platform add android`.
5. **Copy the web build into Cordova**: `npm run build:android` runs the build and copies `dist/` into `android/www/`.
6. **Build the APK**: the same script then runs `cordova prepare` (which re-adds the Android platform on a fresh copy) and `cordova build android`, which uses Gradle and the Android SDK to package `www/` into `app-debug.apk`.
7. **Install and test**: drag the APK onto the Android emulator, or run `adb install Tell-debug.apk`.

package.json: npm scripts

```
"scripts": {
  "libs": "node scripts/build.js libs",
  "build": "node scripts/build.js",
  "serve": "npm run build && http-server dist -p 8100 -c-1",
  "dev": "npm run libs && http-server web -p 8100 -c-1",
  "build:android": "node scripts/build.js android && cd android && npx cordova prepare && npx
cordova build android",
  "run:android": "node scripts/build.js android && cd android && npx cordova prepare && npx
cordova run android"
},
```

scripts/build.js: copying the website into Cordova

```
function buildAndroid() {
  if (!fs.existsSync(path.join(root, 'android', 'config.xml'))) {
    console.error('✗ android/ Cordova project not found — see README "Android build"');
    process.exit(1);
  }
  clean(androidWww);
  copy(dist, androidWww);
  // Service workers don't run inside Cordova's WebView; the app skips registering it there.
  console.log('✓ dist/ copied into android/www');
}
```

9.2 Cordova configuration

`android/config.xml` sets the app identity and how it behaves on Android. `Orientation = default` allows both portrait and landscape, and the adaptive icon and splash screen use Tell's pink "T".

android/config.xml

```
<widget id="com.krezdot.tell" version="2.0.0" xmlns="http://www.w3.org/ns/widgets"
xmlns:cdv="http://cordova.apache.org/ns/1.0">
    <name>Tell</name>
    <description>
        Tells you when you stop actually studying, from how you scroll, type and wander off.
    </description>
    <author href="https://github.com/krez-dot/tell">
        Tell team
    </author>

    <!-- www/ is the built Ionic web app, copied in by `npm run build:android` -->
    <content src="index.html" />
    <allow-intent href="http://*/*" />
    <allow-intent href="https://*/*" />

    <!-- Allow both portrait and landscape; the layout adapts to each -->
    <preference name="Orientation" value="default" />
    <preference name="BackgroundColor" value="0xfffff5f8" />
    <preference name="AndroidWindowSplashScreenBackground" value="#fff5f8" />
    <preference name="AndroidWindowSplashScreenAnimatedIcon" value="res/icon/splash.png" />

    <platform name="android">
        <icon background="res/icon/background.png" foreground="res/icon/foreground.png"
src="res/icon/icon.png" density="xxxhdpi" />
    </platform>
</widget>
```

9.3 Same functionality on both platforms

Every feature exists on both the website and the Android app. Only a few details are handled differently because of how the Android WebView works, and each one is a single `isCordova()` check in the shared code:

Detail	Website	Android app
Exporting a session as CSV	Downloads a .csv file	Copies the CSV to the clipboard (Android's WebView cannot download files)
Service worker (offline / installable)	Registered	Not needed; files are already inside the APK
Back navigation	Browser back button	Hardware back button: switches to Study, or offers to end the session (Ionic ionBackButton event)
Opening a file	Browser file dialog or drag-and-drop	Android system file picker

The Android build was tested on an Android 15 emulator (Pixel-class device, 1080×2400): opening a PDF with the system picker, restless-scroll detection, the marker button, leaving and returning to the app, the back button, saving to history, and rotating between portrait and landscape.

10. Website Screenshots

The screenshots below are the website running in Microsoft Edge at different window sizes. The sample sessions show realistic data.

10.1 Desktop (1440 × 900)

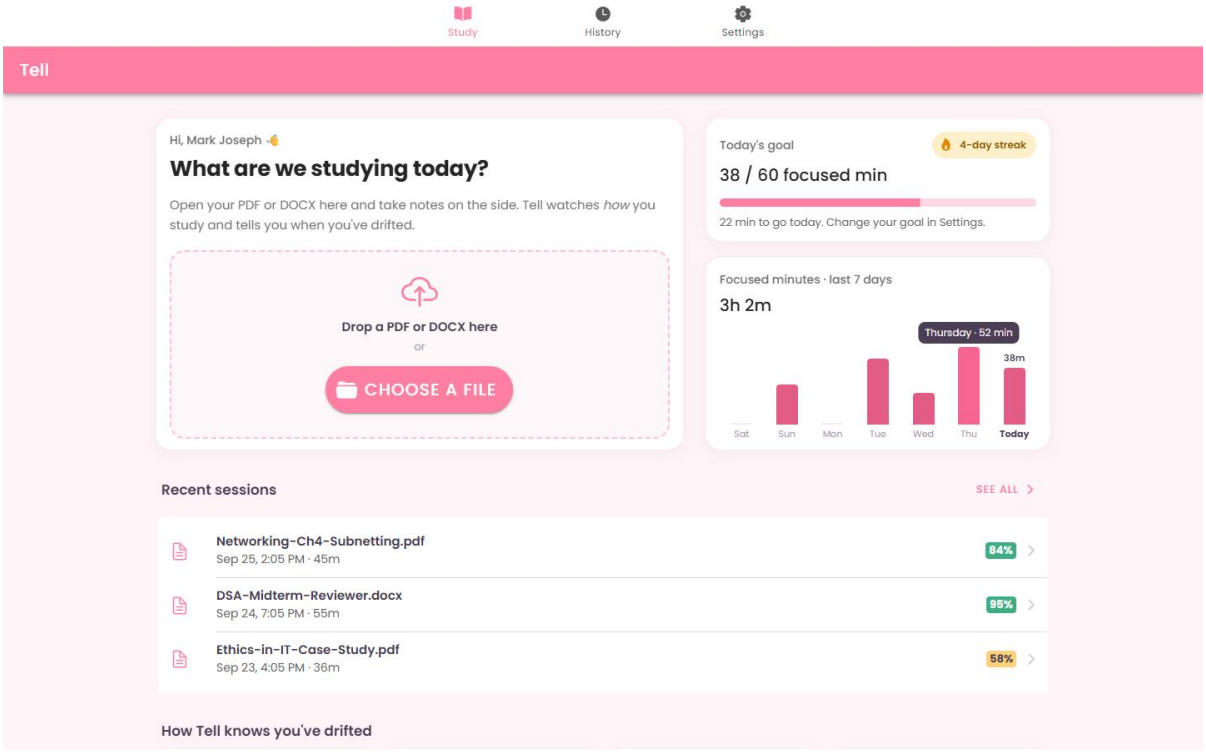


Figure 1. Home: welcome card with drag-and-drop, daily goal with streak, 7-day chart (hovering Thursday), recent sessions

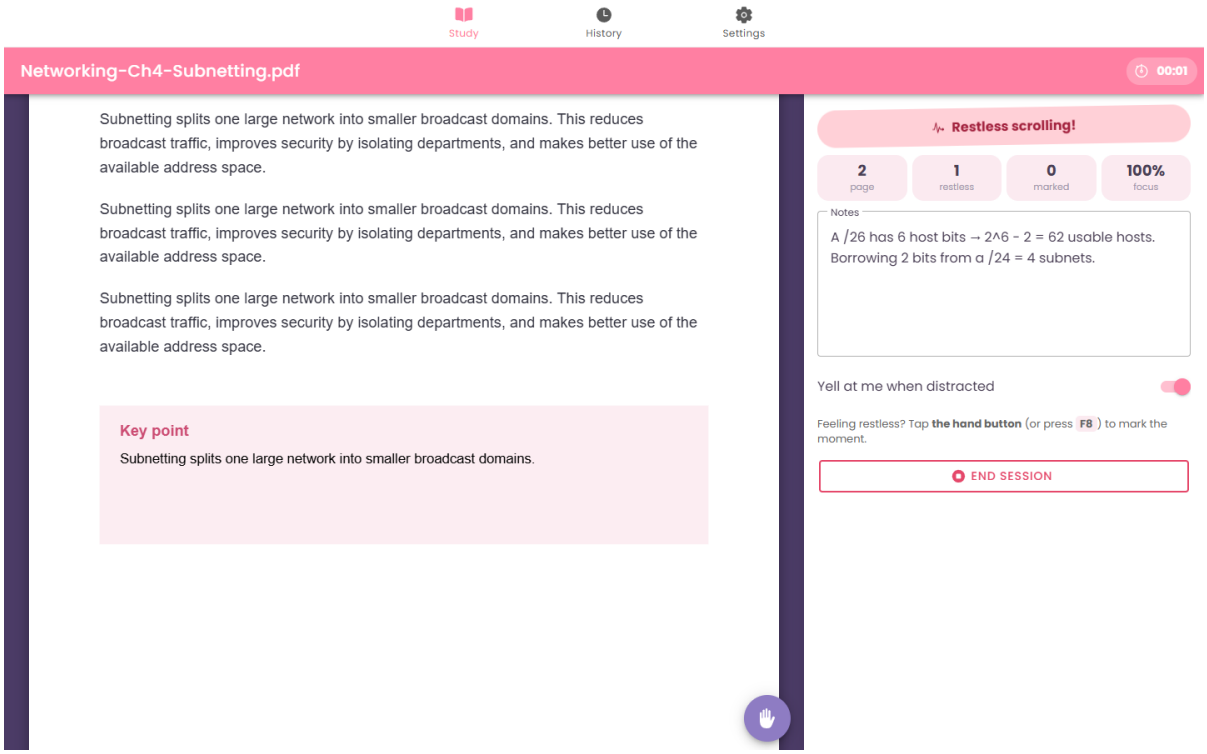


Figure 2. Study session: PDF on the left, live status, stats and notes on the right. Restless scrolling was just detected

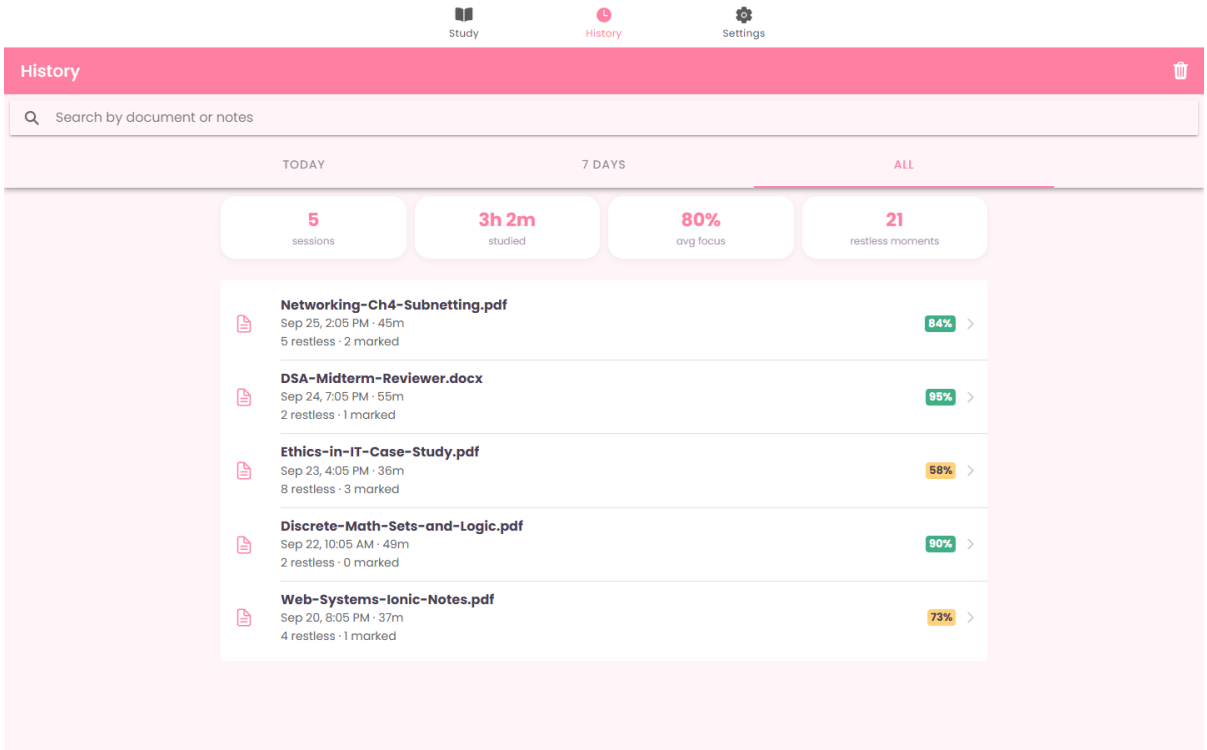


Figure 3. History with search, date filter and totals

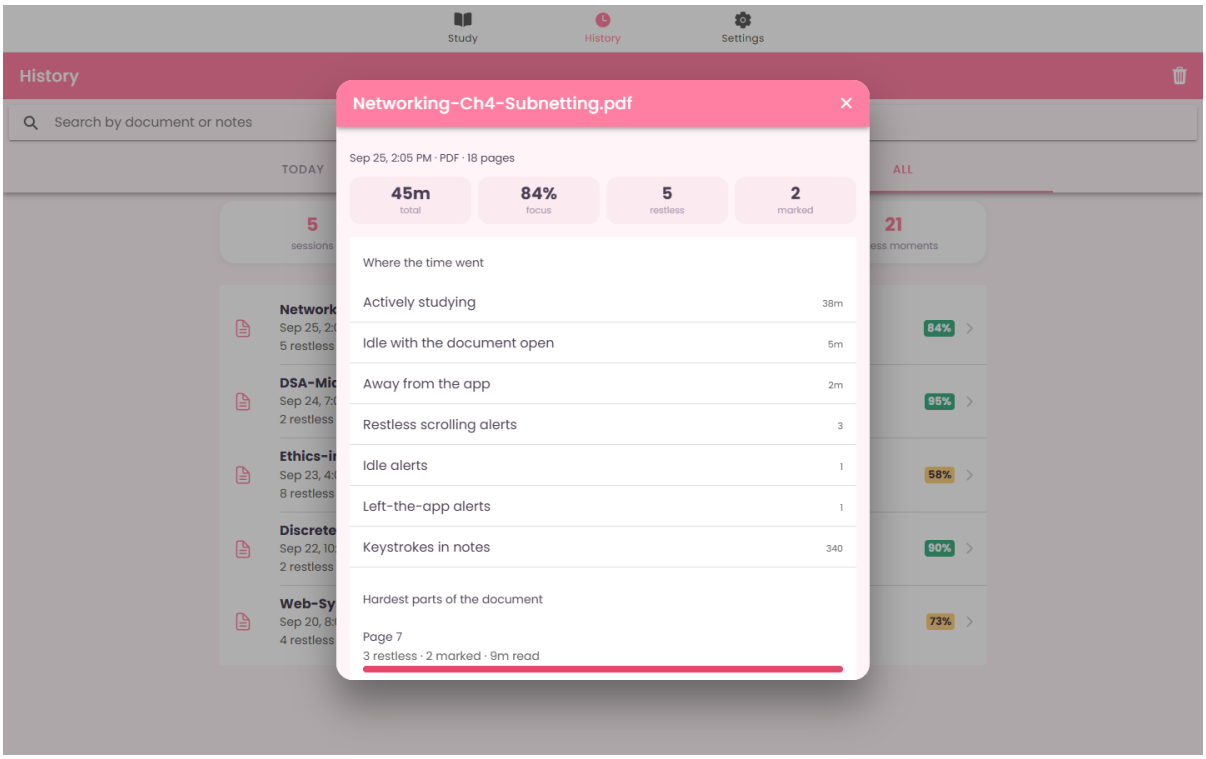


Figure 4. Session details modal: time breakdown and the hardest pages

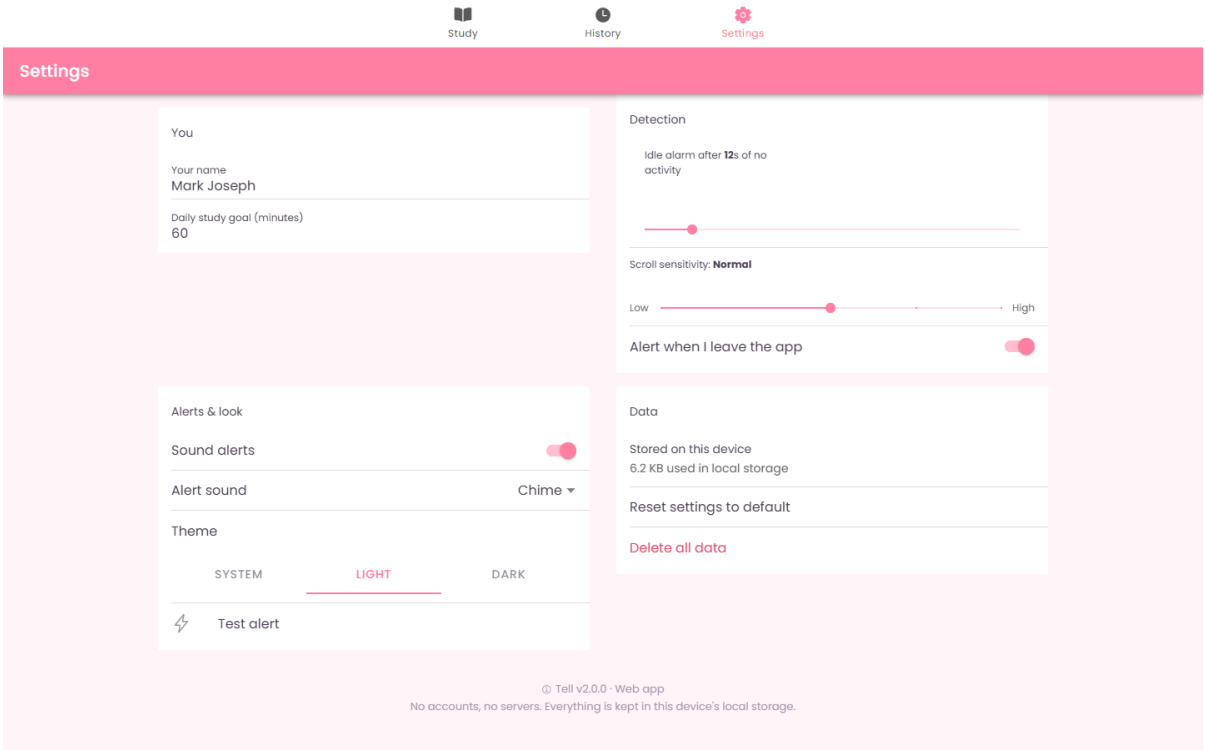


Figure 5. Settings in two columns on a wide screen

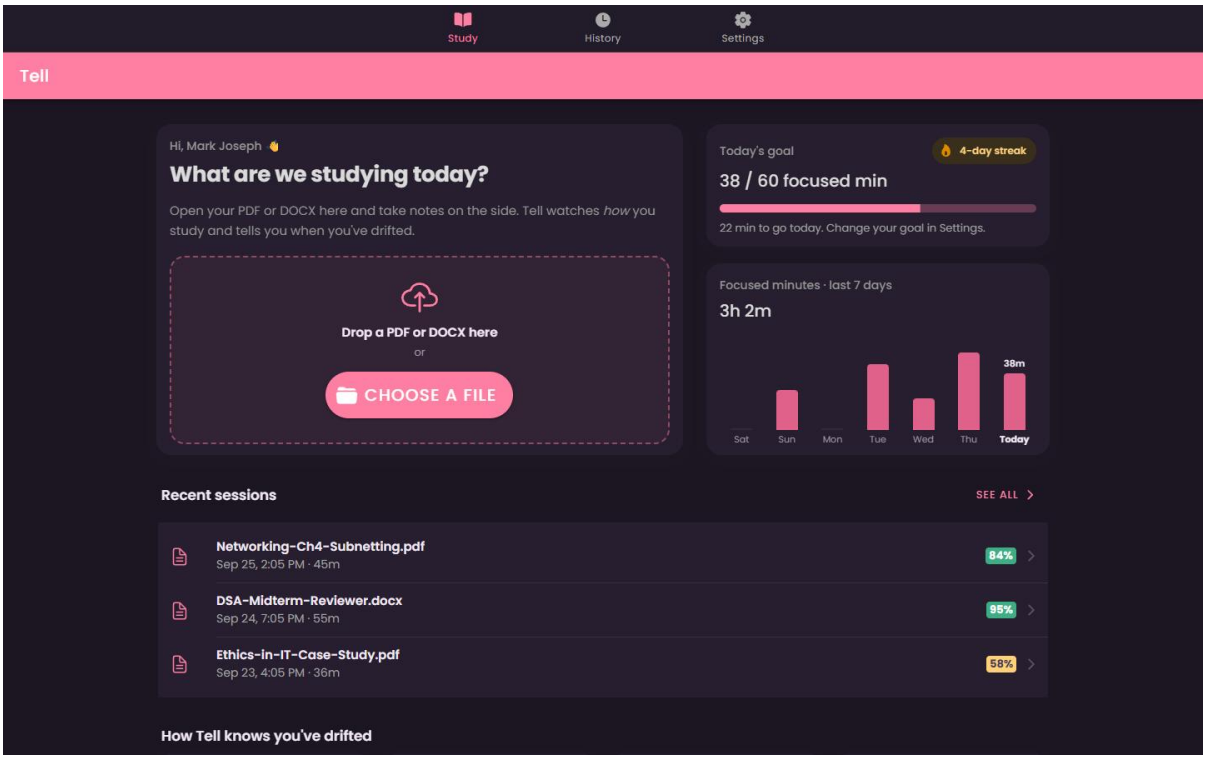
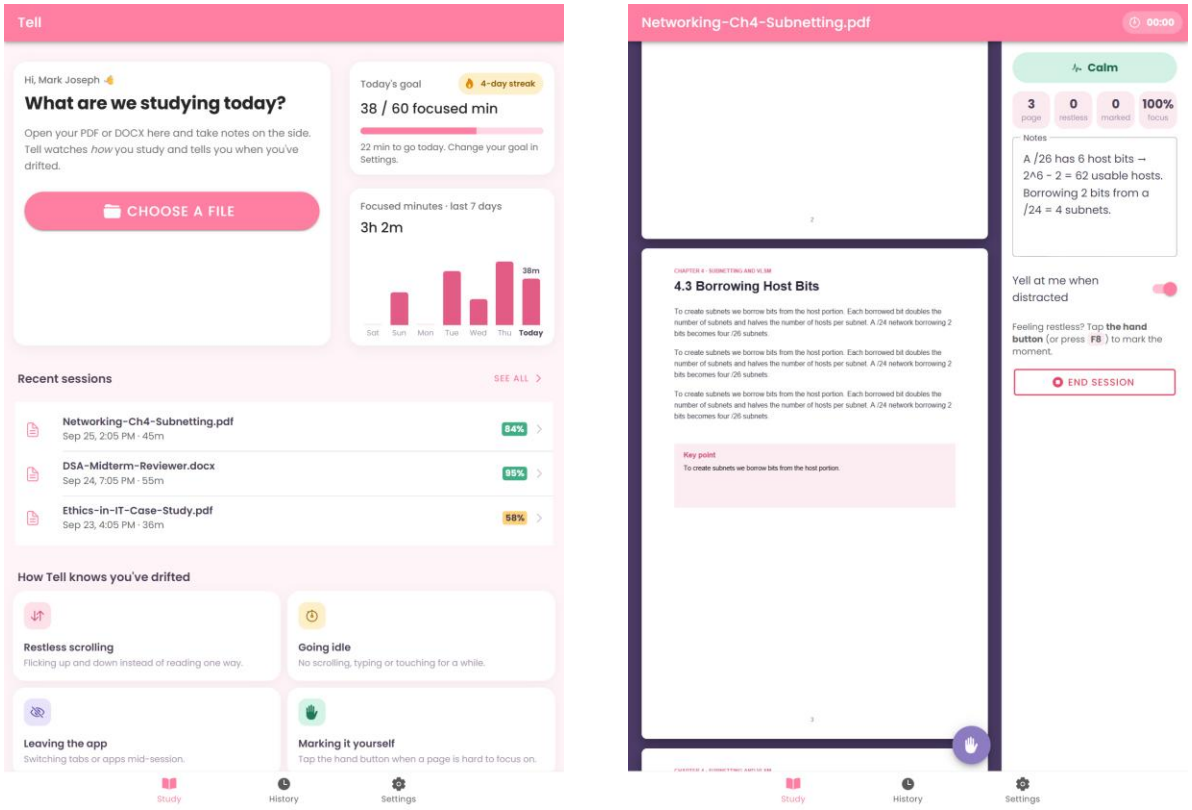


Figure 6. Dark theme

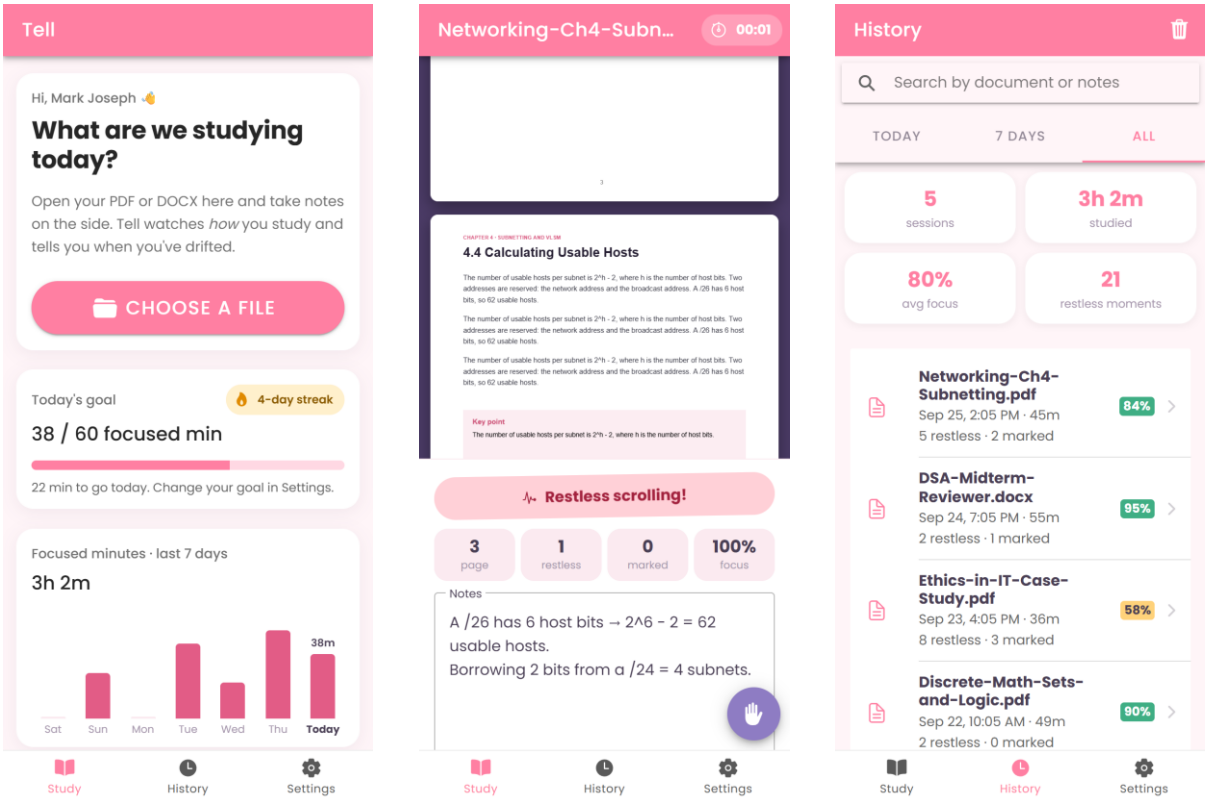
10.2 Tablet (820 × 1180)



Home Study session

Figure 7. Tablet: two-column home, document and side panel side by side

10.3 Phone (390 × 844) and phone landscape (844 × 390)



Home Session (stacked) History

Figure 8. Phone portrait: single column, tab bar at the bottom

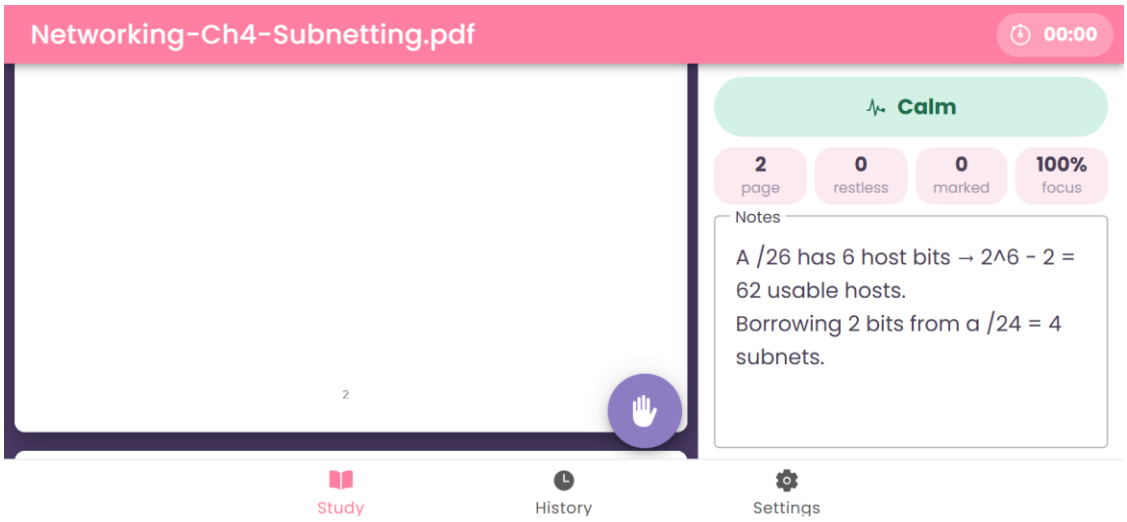
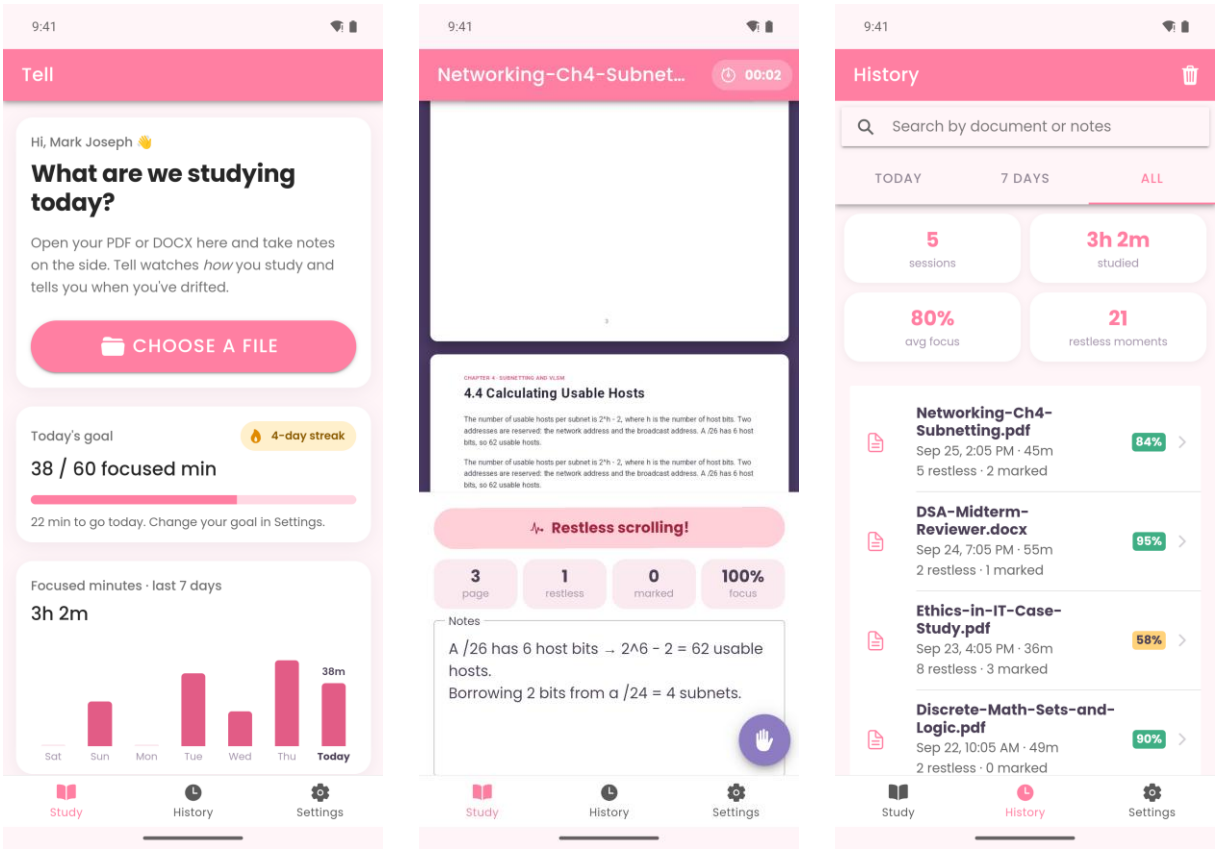


Figure 9. Phone landscape: the session switches to side by side

11. Android Screenshots

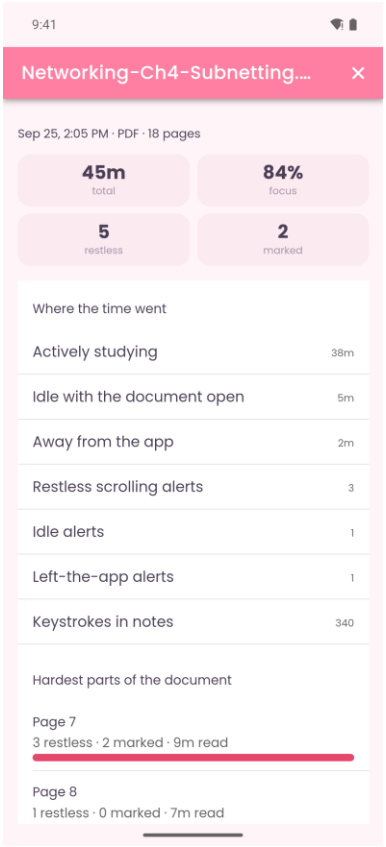
Screenshots of the APK built with Cordova, running on the Android Studio emulator (Android 15, 1080 × 2400). Each screen is shown in portrait and landscape.

11.1 Portrait

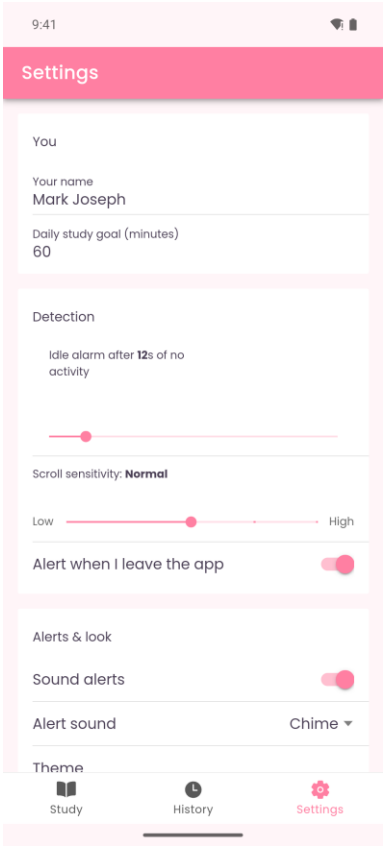


Home | Study session | History

Figure 10. Android portrait



Session details



Settings

Figure 11. Android portrait (continued)

11.2 Landscape

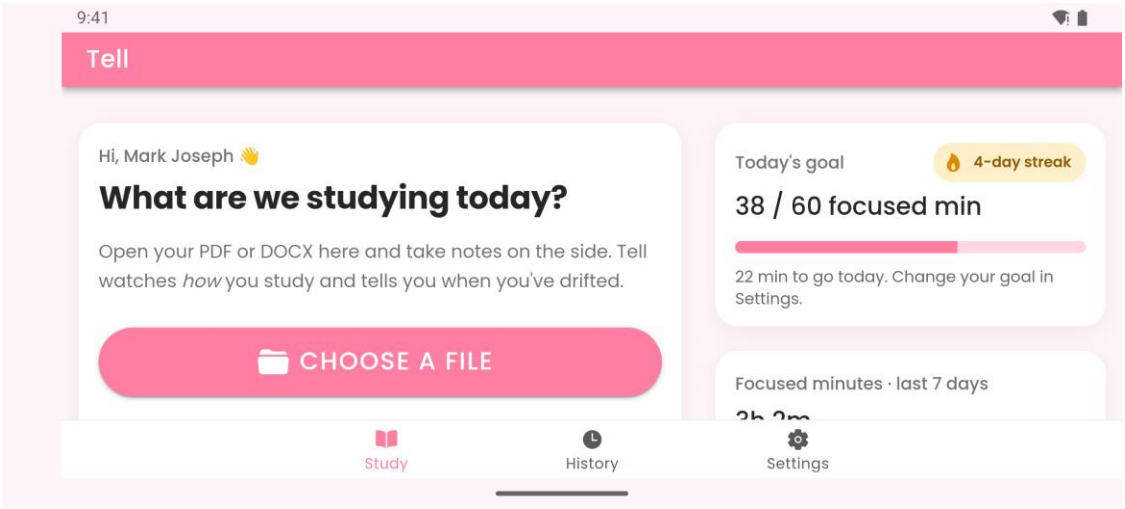


Figure 12. Android landscape: home in two columns

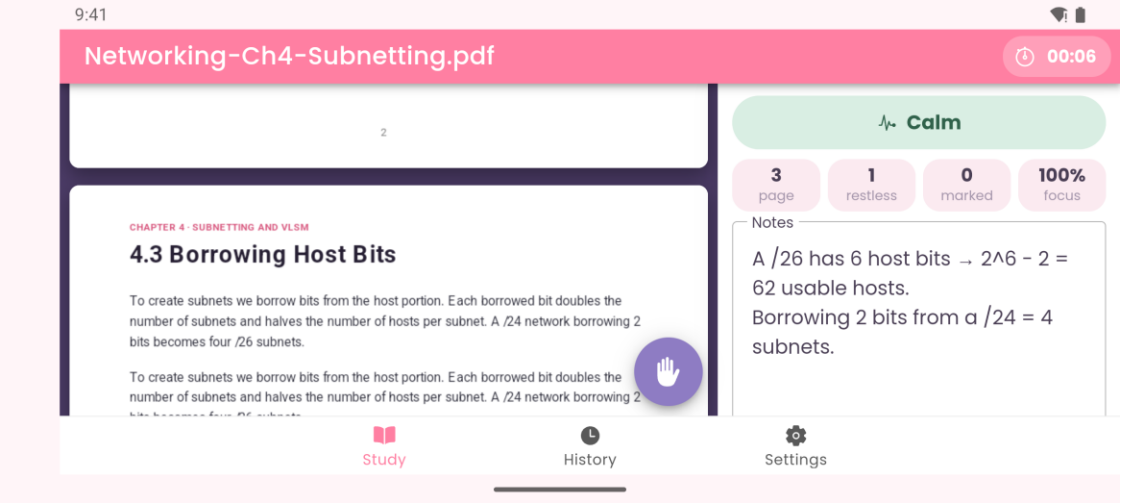


Figure 13. Android landscape: document and side panel side by side

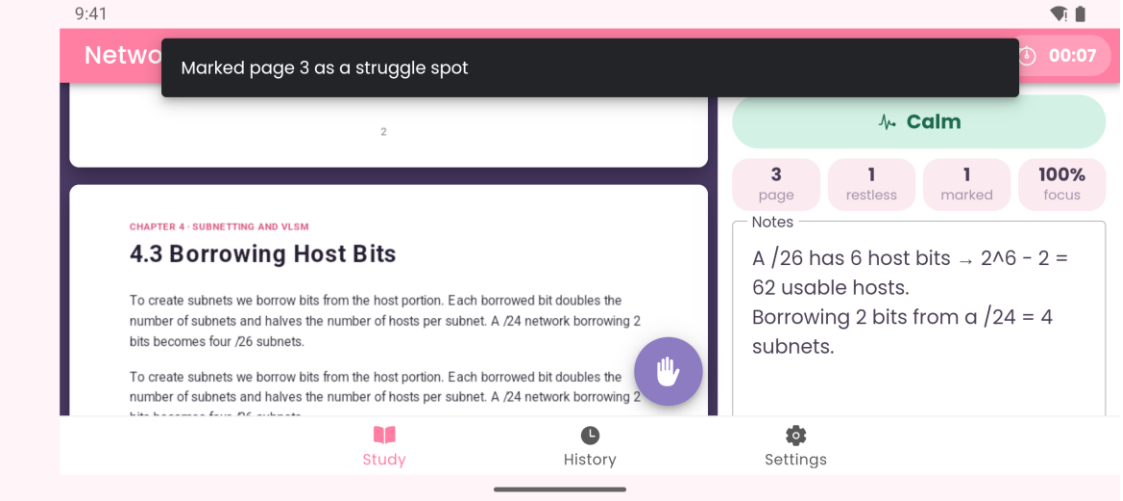


Figure 14. Android landscape: tapping the hand button marks the current page

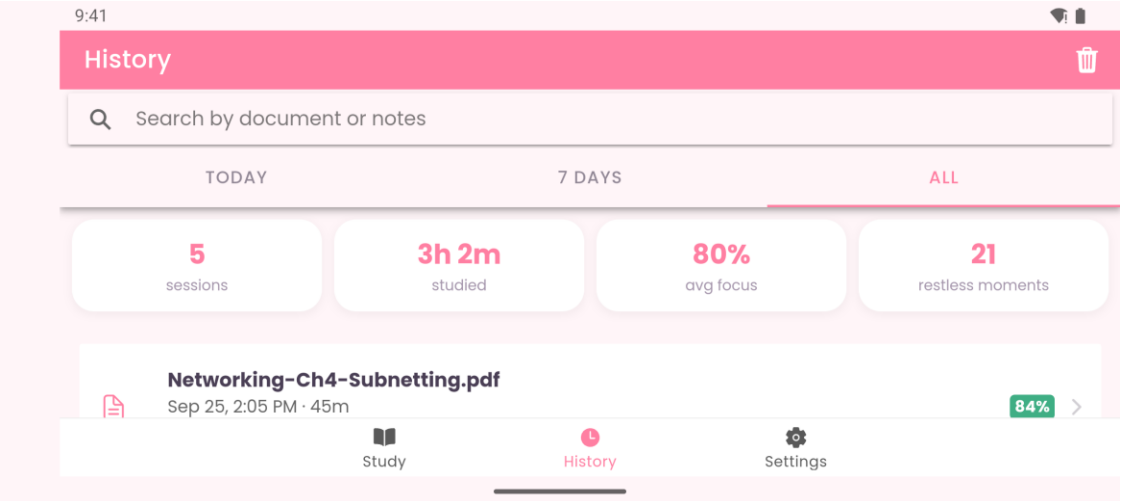


Figure 15. Android landscape: History

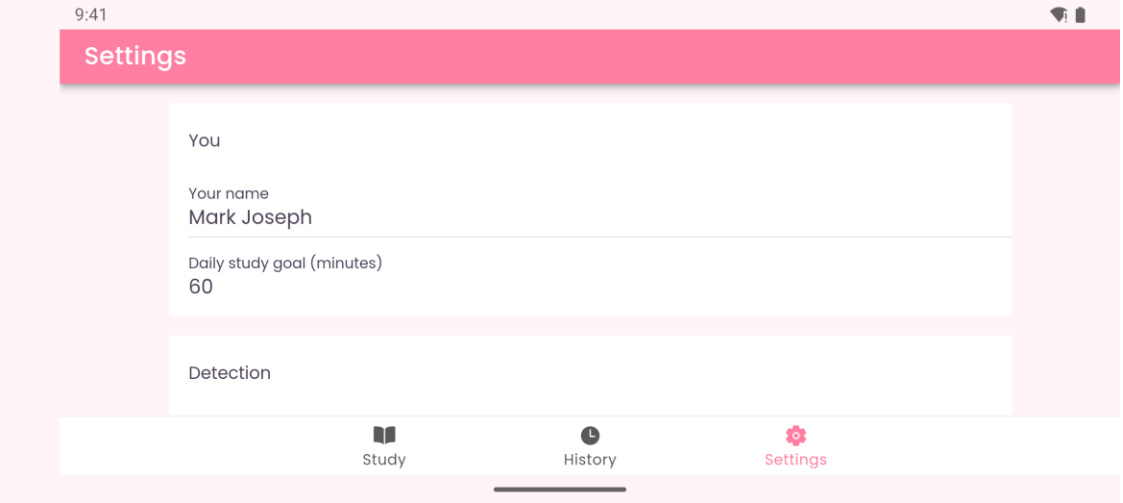


Figure 16. Android landscape: Settings